



Graph Based Protocol

- Graph-based protocols are an alternative to two-phase locking
- This protocol requires prior knowledge about the order in which database items will be accessed
- To acquire such prior knowledge, we impose a partial ordering $\rightarrow$ on the set $\mathbf{D} = \{d_1, d_2, \dots, d_h\}$ of all data items.
 - If $d_i \rightarrow d_j$ then any transaction accessing both d_i and d_j must access d_i before accessing d_j .

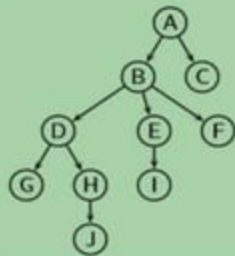


Graph Based Protocol

- A tree or a graph protocol only exclusive locks are allowed. Each transaction T_i can lock data item only once, and must observe the following rules:
 1. The first lock by T_i may be on any data item.
 2. Subsequently, a data Q can be locked by T_i only if the parent of Q is currently locked by T_i .
 3. Data items may be unlocked at any time.
 4. A data item that has been locked and unlocked by T_i cannot subsequently be relocked by T_i



Graph Based Protocol: Example



	T_{10}	T_{11}	T_{12}	T_{13}
1.	lock-X(B)			
2.		lock-X(D)		
3.		lock-X(H)		
4.		unlock(D)		
5.	lock-X(E)			
6.	lock-X(D)			
7.	unlock(B)			
8.	unlock(E)			
9.			lock-X(B)	
10.			lock-X(E)	
11.		unlock(H)		
12.	lock-X(G)			
13.	unlock(D)			
14.				lock-X(D)
15.				lock-X(H)
16.				unlock(D)
17.				unlock(H)
18.			unlock(E)	
19.			unlock(B)	
20.	unlock(G)			



Graph Based Protocol

- **Advantages:**

- The tree protocol ensures conflict serializability as well as freedom from deadlock.
- Unlocking may occur earlier in the tree-locking protocol than in the two-phase locking protocol.
 - shorter waiting times, and increase in concurrency
 - protocol is deadlock-free, no rollbacks are required

- **Disadvantages:**

- Transactions may have to lock data items that it does not access.
 - increased locking overhead, and additional waiting time
 - potential decrease in concurrency



Time Stamp Based Protocol

- Each transaction is issued a timestamp when it enters the system. If an old transaction T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp $TS(T_j)$ such that $TS(T_i) < TS(T_j)$.
- The protocol manages concurrent execution such that the time-stamps determine the serializability order.
- In order to assure such behavior, the protocol maintains for each data Q two timestamp values:
 - **W-timestamp**(Q) is the largest time-stamp of any transaction that executed **write**(Q) successfully.
 - **R-timestamp**(Q) is the largest time-stamp of any transaction that executed **read**(Q) successfully.



Time Stamp Based Protocol

- Use the value of the *system clock* as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
- Use a *logical counter* that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

Time Stamp Based Protocol

- The timestamp ordering protocol ensures that any conflicting **read** and **write** operations are executed in timestamp order.
- Suppose a transaction T_i issues a **read**(Q)
 - If $TS(T_i) \leq \mathbf{W}$ -timestamp(Q), then T_i needs to read a value of Q that was already overwritten. Hence, the **read** operation is rejected, and T_i is rolled back.
 - If $TS(T_i) \geq \mathbf{W}$ -timestamp(Q), then the **read** operation is executed, and $\mathbf{R}$ -timestamp(Q) is set to $\mathbf{max}(\mathbf{R}$ -timestamp(Q), $TS(T_i))$.



Time Stamp Based Protocol

- Suppose that transaction T_i issues **write**(Q).
 - If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced. Hence, the **write** operation is rejected, and T_i is rolled back.
 - If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, this **write** operation is rejected, and T_i is rolled back.
- Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

Time Stamp Based Protocol: Example

- To illustrate this protocol, we consider transactions T_1 and T_2 . Transaction T_1 displays the contents of accounts A and B:
 - T_1 :
 - read(B);
 - read(A);
 - display(A + B);
 - T_2 :
 - read(B);
 - B := B - 50;
 - write(B);
 - read(A);
 - A := A + 50;
 - write(A);
 - display(A + B);



Time Stamp Based Protocol

- In presenting schedules under the timestamp protocol, we shall assume that a transaction is assigned a timestamp immediately before its first instruction. Thus, in schedule 3, $TS(T_1) < TS(T_2)$, and the schedule is possible under the timestamp protocol.
- Schedule 3

T_{14}	T_{15}
read(B)	read(B) $B := B - 50$ write(B)
read(A)	read(A)
display(A + B)	$A := A + 50$ write(A) display(A + B)